

Project 9: Distinct Elements

1. Introduction

This project addresses the "Distinct" problem, where the objective is to determine the number of unique values within a given zero-indexed array A consisting of N integers [cite: 176, 177].

2. Problem Constraints

- Number of elements (N): An integer within the range $[0..100,000]$ [cite: 179].
- Element range: Each element of array A is an integer within the range $[-1,000,000..1,000,000]$ [cite: 180].

3. Algorithm 1: Sorting Approach

3.1 Strategy

This algorithm first sorts the array. In a sorted array, all identical elements are adjacent. By iterating through the sorted array and counting how many times an element differs from its predecessor, we can determine the total number of distinct values.

3.2 Pseudo-code

```
Function GetDistinct_Sorting(A, N):  
    If N is 0: Return 0  
    Sort array A in ascending order  
    Set counter to 1  
    For index i from 1 to N - 1:  
        If A[i] is not equal to A[i-1]:  
            Increment counter by 1  
    Return counter
```

3.3 Complexity Analysis

- **Time Complexity:** $O(N \log N)$ primarily due to the sorting process.

- **Space Complexity:** $O(1)$ or $O(N)$ depending on the specific sorting implementation used (e.g., Heapsort vs. Mergesort).

4. Algorithm 2: Hash Set Approach

4.1 Strategy

This approach utilizes a Hash Set data structure. Since sets inherently store only unique elements, adding all elements of the array to a set automatically filters out duplicates. The final count is simply the size of the set.

4.2 Pseudo-code

```
Function GetDistinct_Set(A, N): Create an empty Hash Set S
For each element x in A: Add x to Set S
Return the size of Set S
```

4.3 Complexity Analysis

- **Time Complexity:** $O(N)$ on average, as insertion into a Hash Set typically takes $O(1)$ time.
- **Space Complexity:** $O(N)$ to store the unique elements in the set.

5. Comparison of Algorithms

Feature	Sorting Approach	Hash Set Approach
Time Complexity	$O(N \log N)$	$O(N)$ (Average)
Space Complexity	$O(1)$ to $O(N)$	$O(N)$

Feature	Sorting Approach	Hash Set Approach
Best Usage	Memory-constrained environments	Performance-critical applications

6. Conclusion

Both algorithms efficiently solve the problem within the given constraints. While the Hash Set approach offers superior time performance, the Sorting approach provides a balanced alternative with potentially lower memory overhead.